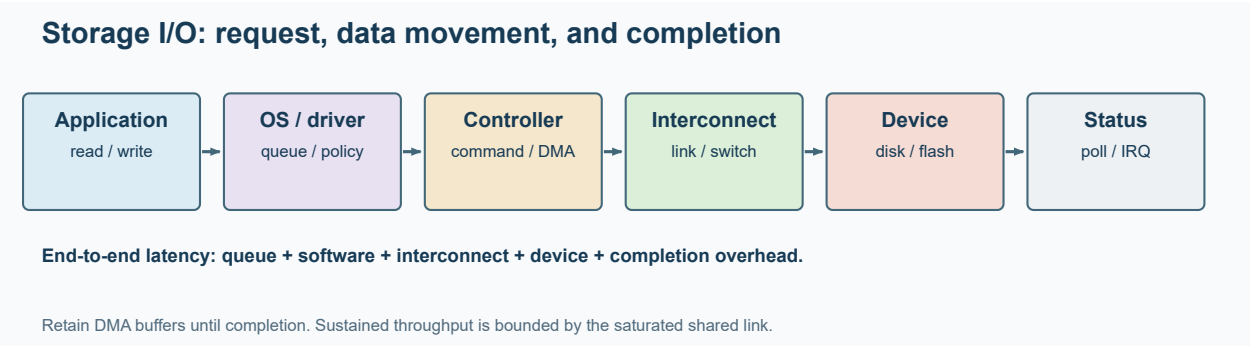
The complete request path



A file read crosses the file system, driver, controller, media, and completion path. Issuing a command, placing data in memory, and allowing the application to use a buffer are distinct events. Fix request size, read/write mix, queue depth, cache state, and completion semantics: a write acknowledged from volatile cache is not necessarily durable after power loss.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Capacity, bandwidth, and IOPS denominators

GB/TB mean $10^9/10^{12}$ bytes; GiB/TiB mean $2^{30}/2^{40}$ bytes. Divide bit/s by eight and account for protocol traffic before claiming payload byte/s.

$$100000\ \mathrm{IOPS} \times 4096\ \mathrm{B} = 409600000\ \mathrm{B/s} = 0.4096\ \mathrm{GB/s}.$$

At 64 KiB per request, the same rate would require 6.5536 GB/s, which the device/link may not sustain. IOPS depends on request size. Response time spans submission to completion; the reciprocal of one request's latency does not establish concurrent throughput.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.1 Workload-specific design objectives

Interactive queries prioritize tail latency; backups need sustained bandwidth and recoverability; transactions also require durable commits and failure atomicity. Identical SSD counts can face different bottlenecks under different access and completion contracts.

Devices, controllers, interconnect, memory, and CPU command processing all impose limits. Added devices consume power, queue entries, interrupt capacity, and recovery time. Peak bandwidth alone guarantees neither low latency nor availability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.2 Failure, reliability, and availability

Failure means service deviates from its specification; restoration returns it to compliance. Redundancy can mask a component fault before it becomes a visible service failure.

Reliability R(t) is the probability of uninterrupted correct service through t. Availability is the probability of correct service at an observation time, or its long-run time fraction, allowing repairs. Fast repair improves availability without erasing an earlier interruption from reliability history.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deriving availability from renewal cycles

Let each cycle have nonnegative uptime U and repair duration D. Assume iid cycle pairs and $0 < E[U] + E[D] < \infty$. The law of large numbers gives

$$A_k = \frac{\sum_{i=1}^{k} U_i}{\sum_{i=1}^{k} (U_i + D_i)} \longrightarrow \frac{E[U]}{E[U] + E[D]}.$$

Thus $A = MTTF/(MTTF + MTTR)$. Individual durations need not equal their means, and the ratio does not bound failures per year. With 100000 h MTTF and 4 h MTTR, A=0.9999600016. Expected long-run downtime in 8760 h is $8760(1 - A)60 = 21.02$ minutes. Reducing MTTR to 0.5 h gives 2.63 minutes, saving 18.39 minutes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A mean lifetime is not a failure deadline

Only an additional constant-hazard exponential model gives

$$R(t) = e^{-\lambda t}, \qquad MTTF = 1/\lambda, \qquad P(T \le t) = 1 - e^{-t/MTTF}.$$

For 100000 h MTTF, the one-year failure probability is $1 - e^{-8760/100000} = 8.39\%$. This guarantees neither 100000 h of survival nor at most one yearly failure. Aging, shared power, and environmental damage can invalidate constant hazards or independence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.3 Disk geometry and logical capacity

Tracks, sectors, surfaces, and cylinders form a legacy fixed-geometry model:

$$C = \text{cylinders} \times \text{surfaces} \times \text{sectors/track} \times \text{bytes/sector}.$$

10000 cylinders, four surfaces, 500 sectors/track, and 512 bytes/sector give 10240000000 bytes=10.24 GB=9.5367 GiB. If these are already logical addressable sectors, do not arbitrarily deduct raw gaps/ECC again. Modern LBA/controller translation does not expose this formula as physical geometry.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Components of one random read

Ignoring queues,

$$T_s = T_{seek} + T_{rotation} + T_{transfer} + T_{controller}.$$

At 12000 RPM, a rotation is 5 ms. Uniform random arrival angle gives 2.5 ms mean rotation. With 500 sectors/track, one sector takes 0.01 ms. Seek 4.5 ms and controller 0.10 ms give

$$T_s = 4.5 + 2.5 + 0.01 + 0.10 = 7.11 \text{ ms}.$$

One outstanding unreordered request gives about 140.65 IOPS. More concurrency may change locality/scheduling and average service time, so this is not a universal queue-depth-independent bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Sequential requests amortize positioning

A merged 100-sector request stays on one track and pays no extra setup or rotation. Transfer takes 1 ms:

$$4.5 + 2.5 + 1 + 0.10 = 8.10\ \text{ms}.$$

51200 payload bytes yield 6.321 MB/s. A hundred independent random requests instead take 711 ms in this model. The saving pays positioning once; it does not accelerate rotation 100 fold. Real sequential models must include track crossings, prefetch, and controller overlap.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

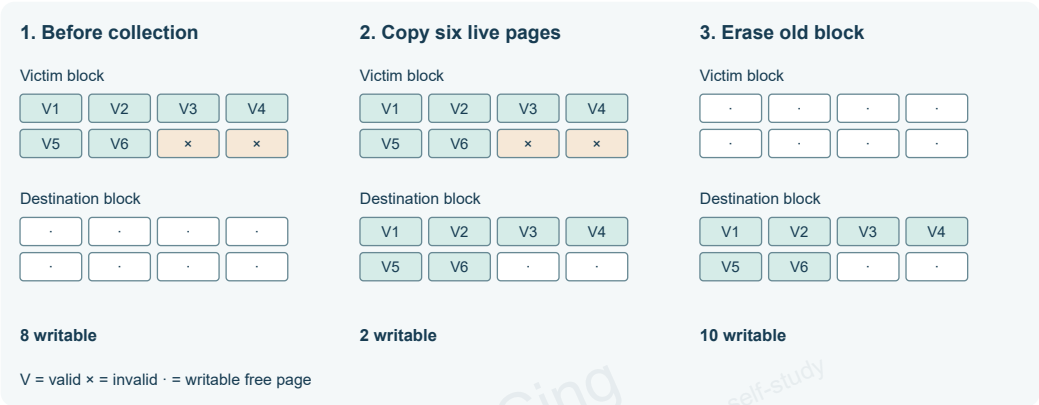
6.4 NAND pages and erase blocks

NOR supports fine-grained reads and selected execute-in-place firmware uses. Denser NAND normally reads/programs pages and erases larger blocks. An FTL maps logical updates to new physical pages, invalidates old pages, and garbage-collects erase blocks instead of overwriting arbitrary words like DRAM.

An eight-page erase block with six valid and two invalid pages requires copying six survivors before erase. The eight reclaimed positions minus six survivor positions leave two net host-update positions. Two host pages plus six copied pages cause eight programs, so WA=8/2=4 for this cycle. Erases are not directly added as written bytes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Garbage collection: trace the pages



The destination starts with eight free pages. Copying six live pages leaves two; erasing the victim yields ten, a net gain of two. Two subsequent same-size host writes plus six copies give batch $WA = 8/2 = 4$. Actual WA depends on workload and metadata.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Endurance estimates and their limits

$$WA = \frac{\text{media bytes written}}{\text{host bytes written}}.$$

120 GB host and 384 GB media giveWA 3.2. With 400 GB physical NAND, 3000 P/E cycles, uniform wear, fixed WA, and no other reserves,

$$400(3000)/3.2 = 375000\ \text{GB} = 375\ \text{TB}$$

of host writes is the rough budget. It is not a warranty. Bad-block reserves, temperature, retention, metadata, and wear leveling matter. Overprovisioning usually helps garbage collection, but a simultaneous working-set change confounds its effect on WA.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.5 Links, buses, and bottlenecks

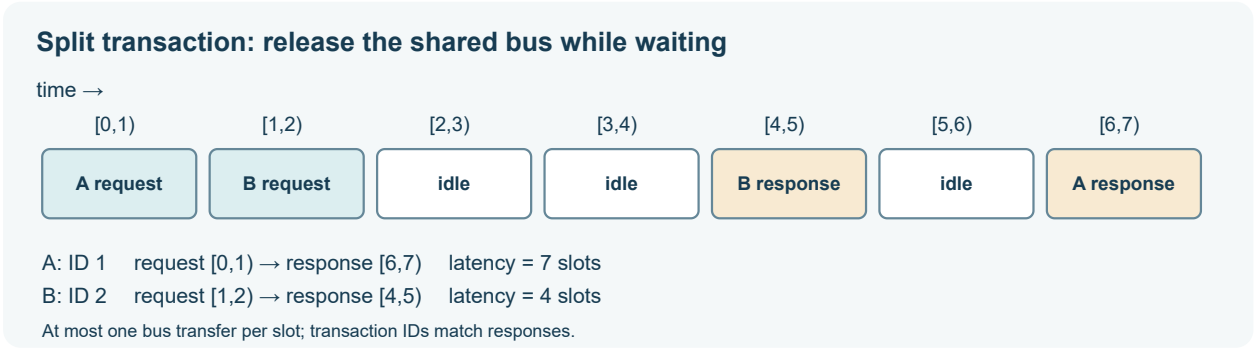
A shared bus serializes competing devices. Point-to-point links/switches allow independent paths but can share upstream links, controllers, and memory channels. Device peaks cannot simply be summed into application throughput.

$$X \le \min(6(80000), 120000, 300000, 150000) = 120000 \text{ IOPS}.$$

The terms represent SSDs, HBA, memory, and CPU/driver. Two extra SSDs leave the HBA bound unchanged. Raising HBA capacity to 240000 exposes the 150000 CPU limit. These are same-size, same-mix capacity bounds; queues, locks, and recovery may lower actual rates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Split transactions release the bus while waiting



Each request or response occupies one slot; the bus carries one at a time, and the device accepts two outstanding requests. IDs match reordered responses. Four of seven slots carry transfers; A takes seven slots and B four. Overlap **does not guarantee lower individual latency**; arbitration and contention matter.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.6 Commands, notifications, and movement

MMIO maps device registers into privileged address space. Some ISAs also provide special I/O instructions. Device cacheability/ordering follows platform rules rather than ordinary RAM assumptions.

Polling/interrupts determine how completion is observed; programmed I/O/DMA determine who moves data. Either completion method can accompany DMA. A doorbell store announces descriptors only after their data becomes device-visible under the DMA/I/O API's required ordering.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Polling and interrupt costs

At 3 GHz and 20000 I/O/s, exactly one 100-cycle poll per request costs 2000000 cycles/s=0.0667% CPU. A 1500-cycle interrupt per request costs 30000000 cycles/s=1%. Merely adding 100 runnable threads does not change this declared one-poll arithmetic.

If each request instead busy-waits 50 microseconds without shared waiting, 20000 requests consume one whole core-second. The earlier 0.0667% no longer applies. A hybrid briefly polls then sleeps; coalescing and completion queues batch work. Compare setup, busy waiting, switches, batch size, and latency distributions.

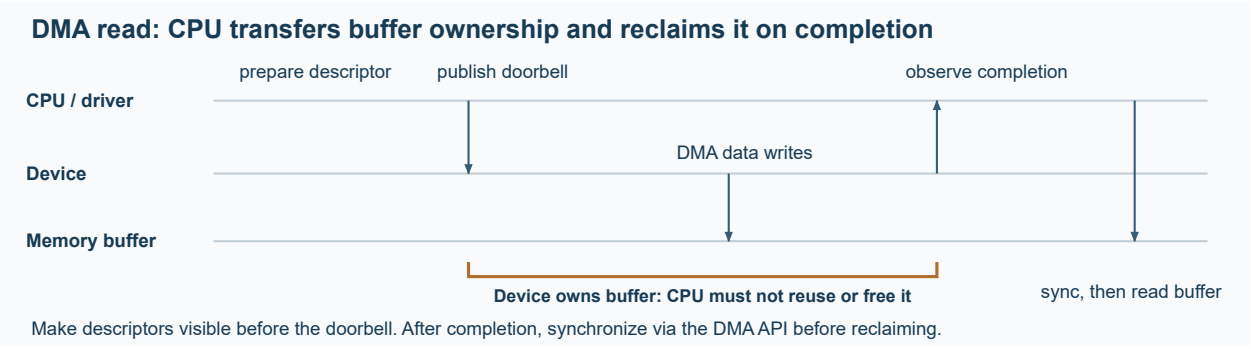
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Interrupt priority and completion processing

The device sets pending state; enables/priorities determine acceptance. The driver identifies the source, reads status, processes completions/errors, acknowledges according to device rules, and wakes waiters. A shared interrupt may belong to another device.

A low-priority handler masking all interrupts for 8 microseconds makes a high-priority event arriving after 1 microsecond wait at least 7 microseconds. Nesting requires separate saved context, protected shared state, and bounded depth. Coalescing lowers handler count but can delay the first completion notification.

DMA buffer ownership



CPU maps the buffer and publishes descriptors. Do not free, reuse, or read it during DMA; after completion, synchronize visibility using the DMA API. On noncoherent systems, a late dirty-line writeback can overwrite DMA data. Timeout alone does not prove DMA stopped; ensure writes cease before unmapping.

DMA saves core copying work

$$64(2^{20})/0.020 = 3.3554432\ \mathrm{GB/s}.$$

A 500-cycle setup at 3 GHz takes 0.1667 microseconds. Programmed I/O takes 16 copy cycles plus 10 transaction cycles per 16 bytes:

$$(64(2^{20})/16)(26) = 109051904\ \mathrm{cycles},$$

or 36.35 ms of CPU execution before waits. DMA still consumes interconnect, memory, IOMMU, and completion work; setup time is not end-to-end transfer time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.7 Little's law

For stable flow with finite averages, mean requests in the system L, completion rate lambda, and response W satisfy

$$L = \lambda W.$$

The area under the request-count-over-time curve equals the sum of request residence times. Divide by observation length and let residual boundary effects vanish. L includes waiting and service; queue-only measures use Lq/Wq.

100000 IOPS at 80 microseconds mean response implies L=8. Setting queue depth eight does not guarantee that rate, because changing concurrency may change W. The law checks mutually consistent steady-state observations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why M/M/1 response rises near saturation

Assume Poisson arrivals lambda, independent exponential service rate mu, one FCFS server, unlimited waiting room, and rho=lambda/mu<1. Balance gives $p_n\lambda = p_{n+1}\mu$, hence $p_n = p_0\rho^n$. Normalization yields $p_0 = 1-\rho$. Differentiating the geometric series gives

$$L = (1-\rho)\sum_{n\geq 0} n\rho^n = \frac{\rho}{1-\rho}, \qquad W = L/\lambda = \frac{1}{\mu-\lambda}.$$

Unchanged mean service can coexist with sharply rising queueing. At rho≥1, this finite stationary mean does not exist. A negative denominator cannot be treated as ordinary latency. The model does not assert that actual disk service is exponential.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One device under different loads

For mu=200 IOPS, mean service is 5 ms.

Arrival rate	Utilization	Response W	Queue Wq	Requests L
100	0.5	10 ms	5 ms	1
160	0.8	25 ms	20 ms	4
190	0.95	100 ms	95 ms	19

Going from 160 to 190 increases completions 18.75% while response quadruples. This follows the declared model, not every device. Specify mean/tail latency limits alongside maximum IOPS.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.8 Fixed costs and durability

Serial nonoverlapped latency components add; resource throughput bounds take a minimum. CPU/device work may overlap partially, so neither unconditional addition nor perfect overlap is justified.

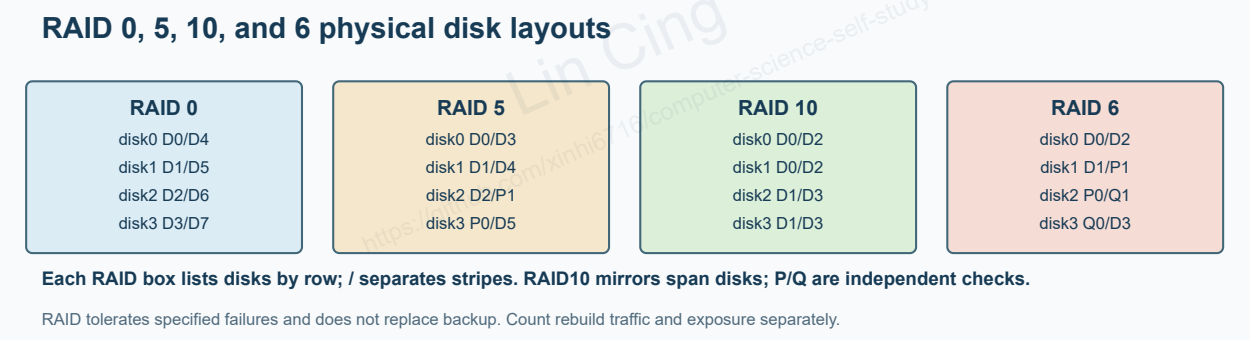
Compare identical completion endpoints. A RAM acknowledgement and a durable-media acknowledgement are different services. Protected caches or explicit flush completion may define durability. Retry semantics also depend on partial completion:repeating a write may be safe, while repeating an increment can apply it twice.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.9 RAID capacity and failures

Assume n equal C-capacity disks, no spares/metadata/reserves, and known whole-disk erasures with correct survivors and working controllers/power.

Layout	Usable capacity	Recoverable failure
RAID0	nC	no redundancy
paired RAID1/RAID10	nC/2, n even	at most one per mirror pair
RAID4/5	(n−1)C	any one disk
RAID6 P+Q	(n−2)C	any two under the encoding assumptions



RAID3 fine-grained striping uses synchronized access. RAID4 block striping distributes small reads but concentrates parity writes. RAID5 rotates parity, removing the fixed parity-disk bottleneck without eliminating parity work.

XOR recovery proof and example

Let $P = D_0 \oplus D_1 \oplus D_2$. Associativity, commutativity,$x \oplus x = 0$, and $x \oplus 0 = x$ give

$$P \oplus D_0 \oplus D_2 = D_1.$$

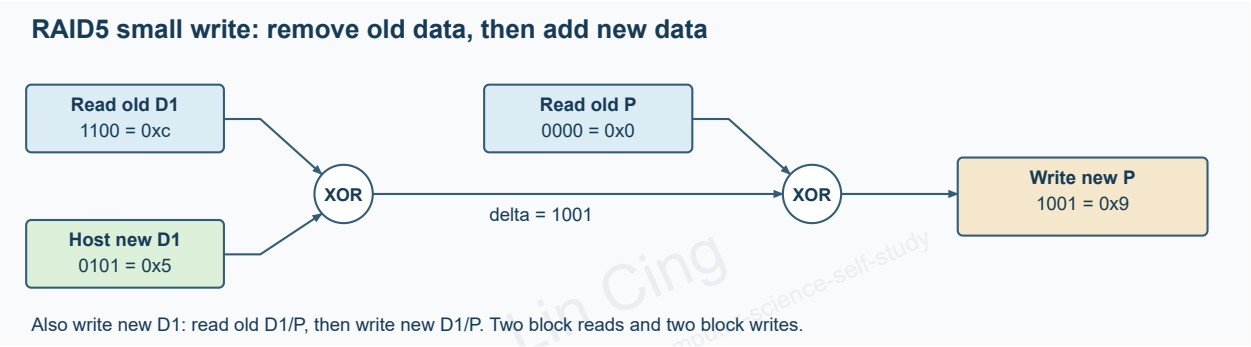
For 1010, 1100, 0110, parity is 0000. Recovering D1 computes 0000 XOR 1010 XOR 0110=1100. Two unknown blocks leave one equation and generally multiple solutions. Silent wrong values have unknown locations; recovering a known erasure does not locate arbitrary corruption.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Small writes and full stripes

$$P' = P \oplus D_{old} \oplus D_{new}.$$

Reading old data/parity and writing new data/parity costs two reads/two writes. Changing 1100 to 0101 gives parity 0000 XOR 1100 XOR 0101=1001.



A full m-data-block stripe gets all new data from the host and writes m+1 blocks without old reads. For k updates, RMW costs 2 k+2 operations; reconstruct-write reads m−k unchanged blocks and writes k+1, total m+1. Compare those workloads with cache state and parallel bottlenecks.

RAID5 read/write mixture bound

Eight disks at 100 block I/O/s provide 800 aggregate operations/s. A small read costs one; a small RMW write costs four. With read fraction r,

$$\text{operations/request} = r + 4(1-r) = 4 - 3r, \qquad X \le 800/(4-3r).$$

At r=0.75, the bound is 457.14 IOPS. Balanced traffic, no combining, and no rebuild are premises. Locks, controller limits, and one overloaded disk can reduce it. RAID4's parity disk alone needs two operations per small write, limiting writes to half its capacity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Duplicate parity is not RAID6

If Q merely copies P, the second equation adds no information. In a field GF(2^w), use distinct nonzero coefficients ai:

$$P = \sum_i D_i, \qquad Q = \sum_i a_i D_i.$$

Field addition is XOR. Removing known survivors for erasures i, j gives $X = D_i + D_j$,$Y = a_i D_i + a_j D_j$. Distinct coefficients ensure $a_i + a_j \neq 0$, so

$$D_j = (Y + a_i X)/(a_i + a_j), \qquad D_i = X + D_j.$$

Division uses a field inverse, not ordinary integer division mixed with XOR.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

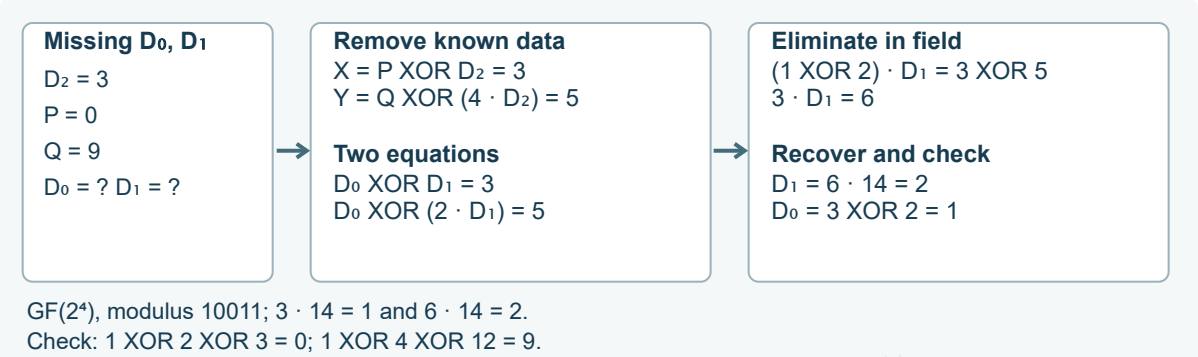
A reproducible P+Q example

Use GF(16) modulo binary 10011, the polynomial $x^4 + x + 1$, with coefficients 1, 2, 4. Multiply by two by shifting; when the old high bit is one, truncate to four bits and XOR 0011.

$D_0 = 1, D_1 = 2, D_2 = 3$ give $P = 1 \oplus 2 \oplus 3 = 0$ and $Q = 1 \oplus 4 \oplus 12 = 9$. If D_0, D_1 disappear, $X = P \oplus D_2 = 3$ and $Y = Q \oplus (4D_2) = 9 \oplus 12 = 5$. Thus $3D_1 = Y \oplus X = 6$. In the stated field modulo binary `10011`, $3 \cdot 14 = 1$, so $3^{-1} = 14$ and $6 \cdot 14 = 2$. Hence $D_1 = 2$ and $D_0 = 3 \oplus 2 = 1$. Actual layouts may use other fields or coefficients; this proves the stated code's recovery.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P+Q: recovering two missing values



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Given D_2, P, and Q, remove D_2's contribution from each parity and XOR the two equations to eliminate D_0. Addition is XOR; multiplication and inverses are in the specified finite field. Ordinary integer division cannot replace field arithmetic.

The parity write hole

Data and parity are separate writes. Power loss after new D1=0101 becomes durable but before old parity 0000 changes leaves P unequal to XOR(data). Correctly readable blocks can still belong to inconsistent versions.

Parity algebra alone supplies no atomic update. Protected write cache, journaling, or stripe transactions need explicit ordering, durability, and restart rules. Redundancy also cannot recover legitimate deletions; it does not replace independent backup.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rebuild traffic denominators

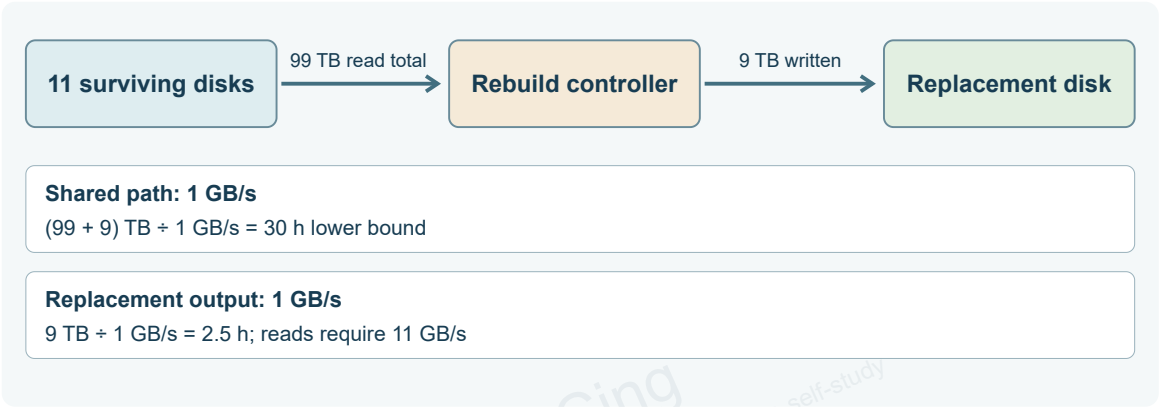
A 12-disk 10 TB RAID5 is 90%full and rebuilds only nine valid TB per disk. Each missing block reads 11 survivors, so reads 99 TB and writes 9 TB. At shared aggregate 1 GB/s,

$$T \geq (99+9)10^{12}/10^{9} = 108000\ \mathrm{s} = 30\ \mathrm{h}.$$

If 1 GB/s instead means replacement output, nine TB takes 2.5 h but needs 11 GB/s surviving reads. Full-disk reconstruction reads 110 TB+writes 10 TB, total 120 TB, and takes at least 33.33 h on the aggregate path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rebuild: match traffic to the bandwidth



Rebuilding 9 TB reads 99 TB and writes 9 TB. A shared path carries both; if 1 GB/s describes only replacement writes, the read path must simultaneously sustain 11 GB/s. These idealized lower bounds omit contention and overhead.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A second failure during rebuild

An eight-disk RAID5 has already lost one disk. Fix repair time r=10 h and independent constant survivor hazards lambda=10^-5/h. All seven survive with probability

$$(e^{-\lambda r})^7 = e^{-7\lambda r}.$$

$$P(\text{another failure}) = 1 - e^{-7(10^{-5})(10)} = 0.000699755 = 0.06998\%.$$

This conditions on degraded entry and one repair interval. It is not annual array data-loss probability. Uncorrectable reads, correlated failures, and variable repair durations require additional models.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.10 Historical Sun Fire and power boundaries

The book's circa 2008 Sun Fire x4150 is a 1 U case with a maximum configuration of two four-core Xeon sockets, 64 GB DRAM, eight 15000 RPM SAS disks, and RAID controller. These historical specifications are not current product claims. A 19-inch rack has 1 U=1.75 inches; 42 U=73.5 inches, subject also to power/cooling limits.

A separate teaching model uses idle 200 W, full 400 W, and 2000 transactions/s, or 0.2 J/transaction. Half load at 300 W and 1000/s costs 0.3 J/transaction. Consolidating two half-loaded machines saves energy only if capacity, tail latency, fault tolerance, and migration costs remain acceptable.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Different workloads on the same historical configuration

A teaching workload, not the book's measurement, uses eight cores each sustaining 10^9 instructions/s. Each 64 KiB I/O consumes 250000 user+150000 OS instructions, giving CPU bound 20000 IOPS.

Eight random-read disks at 4 ms service provide 2000 IOPS. Sequential disks at 112 MB/s each provide $8(112 \times 10^6)/65536 = 13671.875$ IOPS. A 2 GB/s RAID-card payload link provides 30517.578 IOPS.

Ignoring other conflicts, random throughput is bounded by 2000 and sequential by 13671.875, both disk-limited. Faster SSDs exceeding 20000 IOPS can expose the CPU bound. Do not combine a historical configuration figure and this new workload into a product benchmark.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.11 Network transfer and waiting

A 10 Gbit/s link with 80%payload efficiency gives 8 Gbit/s=1 GB/s.

$$64(2^{20})/10^9 = 67.108864 \text{ ms}.$$

Sixteen 4 MiB batches have 15 inter-batch 0.20 ms gaps when the connection is established, total 3 ms, and overall 70.108864 ms. A hundred streams may hide waits but cannot create 100 GB/s on that shared link.

Propagation, serialization, queueing, and protocol setup are distinct. Bandwidth times RTT is in-flight data needed to sustain a rate, not automatic extra payload per request.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.12 Counterexamples to common claims

DMA still costs setup/completion CPU time and memory bandwidth. Two parity blocks require independent encoding and do not solve write holes. High MTTF is a mean rather than a survival deadline.

Equal IOPS at 4 KiB and 64 KiB imply different bytes/s. More concurrency can increase response time, as the M/M/1 table shows. RAID cannot undo deletion or an attacker overwriting all replicas. Each counterexample identifies its violated premise instead of asserting a universal device rule.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.13 Inferring bottlenecks from observations

When throughput stops rising while response grows, measure utilization, queue length, and per-I/O work for each resource. Little's law checks common steady-state accounting but does not localize saturation by itself.

Fast acknowledgements followed by power-loss data loss require a completion/durability investigation. Slow rebuilds require separate survivor-read, replacement-write, and shared-link accounting. Both performance and failure analysis begin with the end-to-end service contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.14 Technology changes, analysis persists

Mechanical positioning, out-of-place NAND updates, and shared links have different costs but still require separate capacity, service, queueing, and failure models. Switches and SSDs change leading costs without removing ownership, completion, or persistence obligations.

Historical cases explain tradeoffs. New comparisons need matching workload and completion data rather than treating 2008 peaks as modern constants.

References and attribution

David A. Patterson and John L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition, Chapter 6. Lin Cing, handwritten computer organization notes; computer organization and structure (CSO) course slides.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study